

Sage Serve

MLOPS - Project

Serving LLMs at Scale

Microsoft 365 serves millions of LLM requests across multiple data centers.

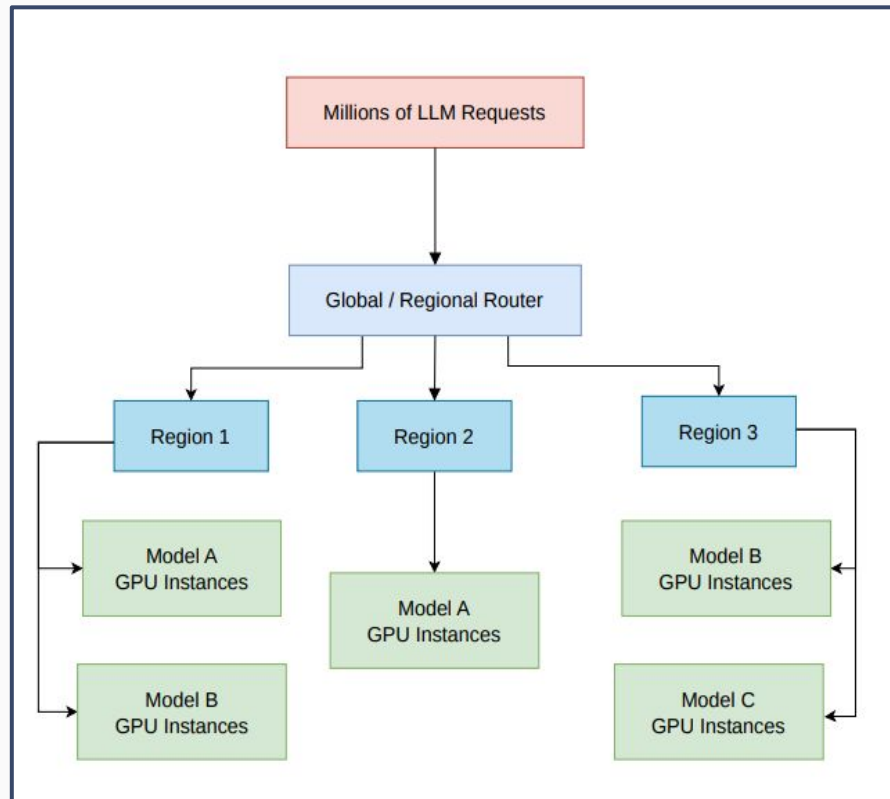
At any point, the system must decide:

- **Which model** should run?
- **Where** should it run?
- **How many GPU instances** should be active?

Note that demand changes over the following:

Time, Region, Model, Workload

GPUs are expensive resources.



Not All Requests Are Equal

Different workloads have very different latency requirements.

Workload	Requirement	Behavior
IW-F	< 1 sec TTFT	Very fast interactive
IW-N	< 1 min TTFT	Normal interactive
NIW	Deadline in hours	Non-interactive

IW workloads are latency sensitive. NIW workloads are flexible.

The First Problem: Reactive Autoscaling

Traditional reactive autoscaling

1. Monitor GPU utilization or incoming load
2. Detect a demand spike
3. Provision a new GPU instance
4. Load the LLM onto the GPU
5. Instance becomes ready

Under-provisioning

GPU isn't ready when demand arrives ,
leads to **SLA violation**

Over-provisioning

GPUs are started in anticipation of demand,
Idle GPU-hours / wasted capacity

The problem is that **LLM instances have long cold-start times**, because provisioning the VM and loading large model weights can take **minutes to hours**.

**Reactive scaling responds to demand;
LLM serving needs to anticipate it.**

Demand Is Predictable

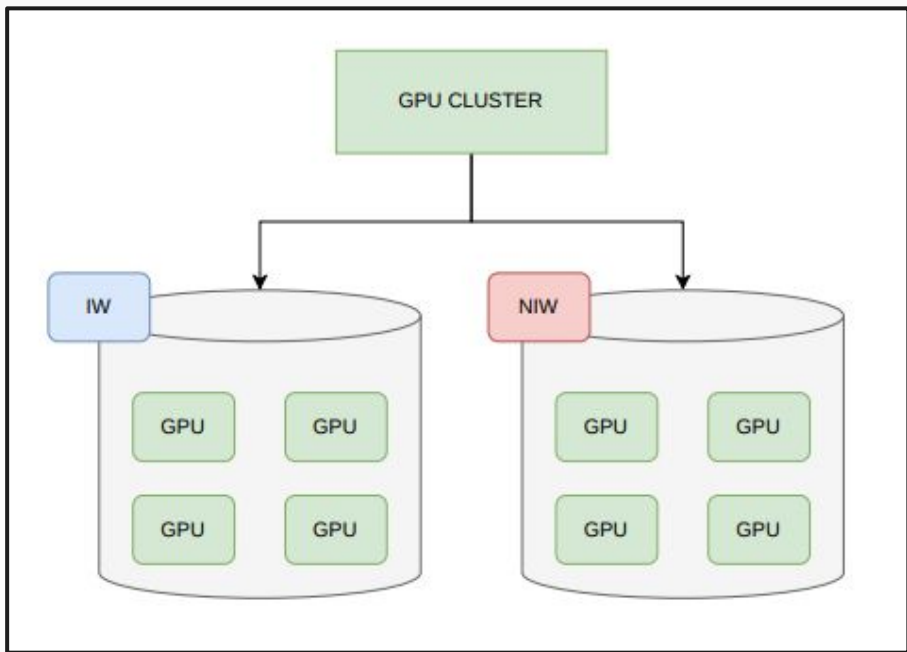
The authors analyze production LLM workloads and find that **demand is not completely random**.

Workload	Requests	Observed periodicity
IW-F	strong temporal patterns	0.7–0.95
IW-N	moderate periodicity	0.3–0.8
NIW	much less predictable	0–0.286

NIW has relaxed deadlines, so it can use spare capacity when available.

The Second Problem: Siloed GPU Pools

Interactive and non-interactive workloads are traditionally served by separate GPU pools.



The problem

Demand is not synchronized across the two pools.

Uneven Demand Example

Interactive demand increases, heavily loading the IW pool, while NIW demand remains low.

One pool can be **overloaded** while another has **unused** GPU capacity

Unified GPU Pool

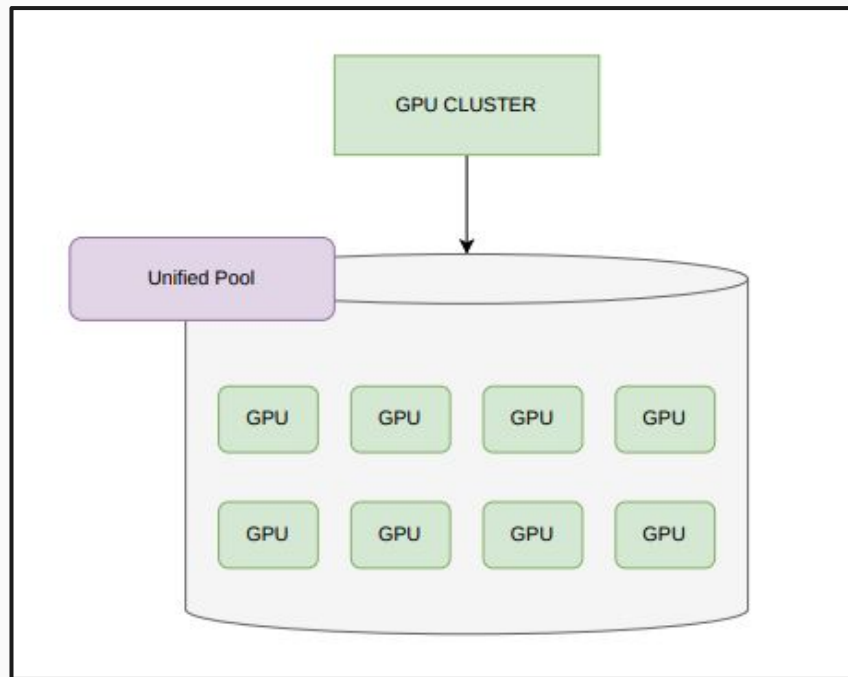
Share GPU capacity while protecting latency-sensitive workloads.

Instead of maintaining independent capacity for each workload, the same GPU pool can be dynamically shared.

IW gets priority
NIW fills available capacity

Experimental Result

34.5% fewer instance-hours



The Capacity Planning Problem

Given the expected workload, how should we provision GPU capacity?

DEMAND

How much traffic
is expected?

CAPACITY

How many
instances are
needed?

SLA

Can requests
meet their
deadlines?

COST

How do we avoid
unnecessary
GPUs?

Find the GPU allocation that satisfies workload SLAs while minimizing resource usage.

ILP Formulation for Routing & Capacity Allocation

Goal

Continuously provision the right number of model instances per region and route requests, to:

- Maximize utility of workload requests completed within SLA
- Maximize GPU capacity utilization for interactive workloads

Objective: minimize wasted resource overheads

$$\arg \min (\gamma + \mu)$$

γ = VM start-up cost

μ = model deployment / weight-loading cost

Subject to: serving $\geq \mathcal{E}$ fraction of forecasted peak TPS per region-model, and cumulative cross-region capacity $\geq$ total forecasted demand per model.

Key Variables

Meaning

$n(i,j,k)$

instances of model i at region j on GPU k

$\rho(i,j)(w)$

forecasted TPS demand, model i , region j , window w

$\theta(i,k)$

TPS throughput of model i on GPU type k

$\delta(i,j,k)$

ILP output — optimal change in VM allocation

$\alpha(k), \sigma(i,k)$

VM acquisition cost & model start-up cost

Solved hourly via ILP. Average solver runtime: 1.41s ($l=4, r=3, g=1$) $\rightarrow$ 33s at ($l=20, r=20, g=5$). Acceptable for hourly decisions.

Formulating Capacity Allocation as an ILP

Goal: Maximize GPU utilization while guaranteeing latency SLAs across multiple data centers.

What we are solving for ($\delta_{i,j,k}$):

- The integer change (+/-) in VM count for model i , per region j , running on GPU k .

Objective Function ($\arg \min(\gamma + \mu)$):

- **Minimize γ (VM Startup Cost):** Expense of acquiring new GPU Virtual Machines.
- **Minimize μ (Model Cold-Start Penalty):** Wasted GPU time spent copying model weights to memory.

This is a simplified overview of the actual ILP

Formulating Capacity Allocation as an ILP

Optimization Constraints & Routing Logic

Local Region Capacity (ϵ -Fraction Rule):

- Each region *must* serve at least an ϵ fraction of its peak local load to ensure low latency.

Global Capacity Coverage:

- Cumulative GPU capacity across all data centers must meet 100% of the forecasted global demand.
- Excess local traffic $(1 - \epsilon)$ is dynamically rerouted to regions with spare headroom.

Physical Bounds:

- System cannot deallocate more instances than currently running ($\delta_{i,j,k} \geq -n_{i,j,k}$).

Formulating Capacity Allocation as an ILP

THE LOGIC

- Instead of manually tweaking server counts, they formulate capacity scaling as an Integer Linear Program solved every hour.
- The objective function balances two main costs mentioned in the initial slide.
- To keep latencies tight, each region handles most of its traffic locally based on a parameter ϵ , but if a regional spike occurs, the ILP accounts for cross-region rerouting to absorb the overflow.
- Because it's an ILP, it solves in about 1.4 seconds in practice.

Need to continuously ensure that the right number of model instances of different model types are provisioned as endpoints at the regions

SageServe: Architecture

Load Predictor

ARIMA forecasts input TPS per model/region every hour.

Queue Manager (NIW)

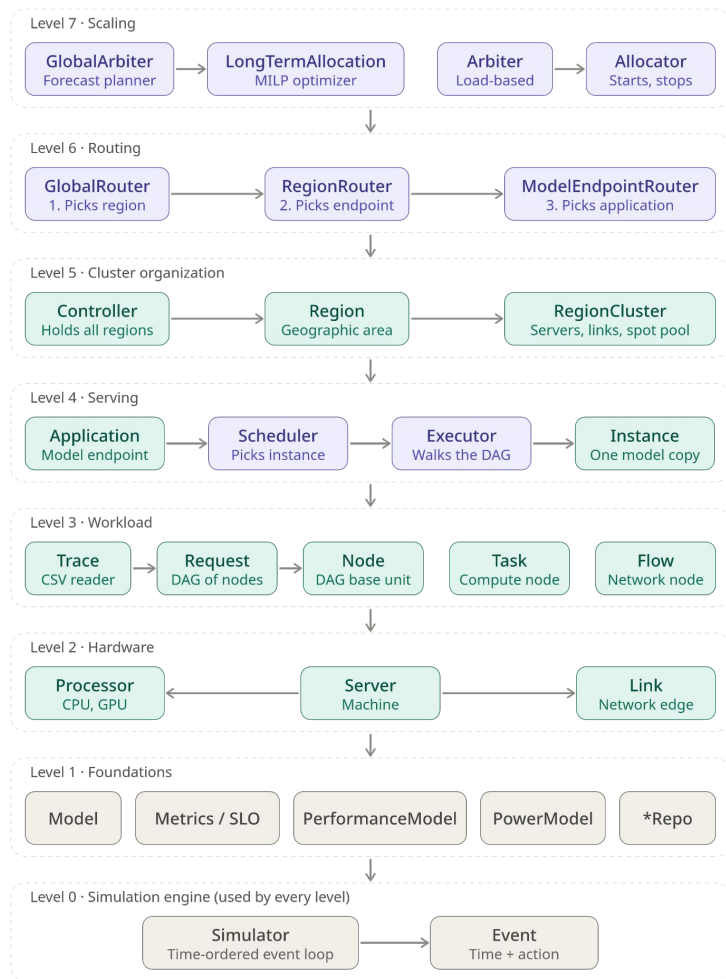
Holds NIW requests and releases them to endpoints when utilization drops below 50–60%.

Autoscaler / Arbiter

Uses the previous ILP formulation for scaling actions: Immediate (LT-I), Utilization-deferred (LT-U), or Utilization+ARIMA-gap (LT-UA).

Request Scheduler

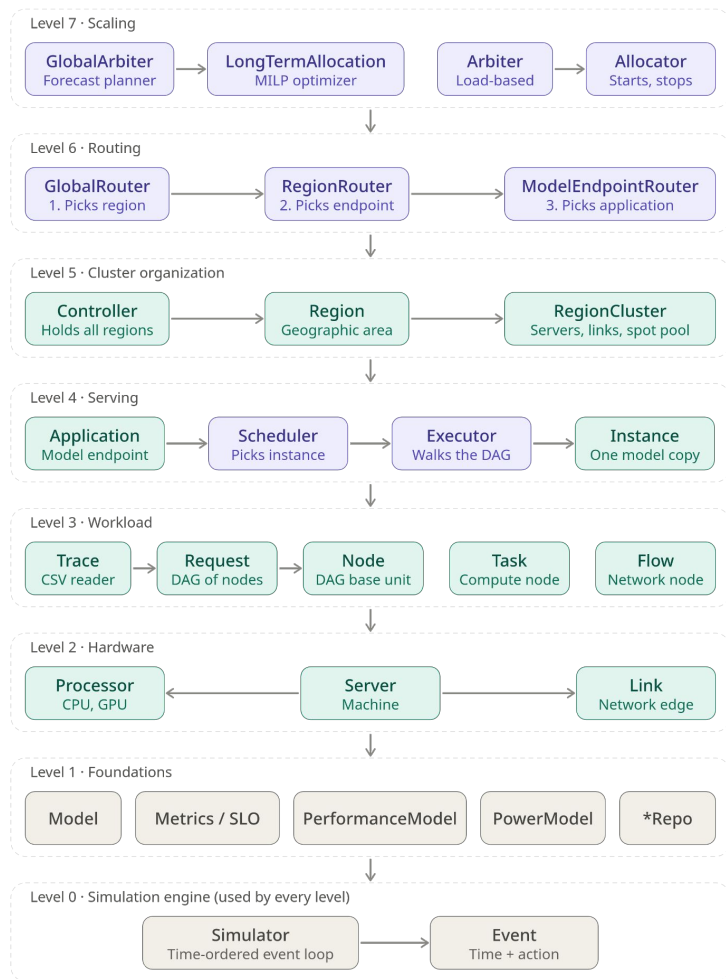
Various Scheduling algorithms: FCFS, EDF, Priority-First (PF), or Deadline and Priority Aware (DPA).



■ Decision logic (routing, scaling)
 ■ Simulated system
 ■ Foundations

Down arrow: uses the level below. Side arrow: main link inside a level.

Group	Mathematical formulation
Request Generation	$\text{Trace} \rightarrow \{\text{Request}_1, \text{Request}_2, \dots, \text{Request}_n\}$
Request Structure	$\text{Request}_i \rightarrow G_i = (V_i, E_i)$, where $V_i = \{\text{Tasks}\}$ and $E_i = \{\text{Dependencies}\}$
Computation	$\text{Task} \rightarrow \text{CPU/GPU} \rightarrow \text{Execution}$
Communication	$\text{Flow} \rightarrow \text{Network Link} \rightarrow \text{Destination Server}$
Request Execution	$\text{Request} \rightarrow \text{Scheduler} \rightarrow \text{Instance} \rightarrow \text{Executor} \rightarrow \text{Tasks/Flows}$
Hardware	$\text{Server} = \{\text{CPU}, \text{GPU}, \text{Network Interface}\}$
Performance	$\text{Execution Time} =$ $f(\text{Model}, \text{Batch Size}, \text{Hardware}, \text{Workload})$
Power	$\text{Power Consumption} = f(\text{Hardware}, \text{Utilization})$
SLO	$\text{Request Latency} \leq \text{Latency Target} \Rightarrow \text{SLO Satisfied}$
Simulation	$\text{System State}_{t+\Delta t} = F(\text{System State}_t, \text{Event at } t)$

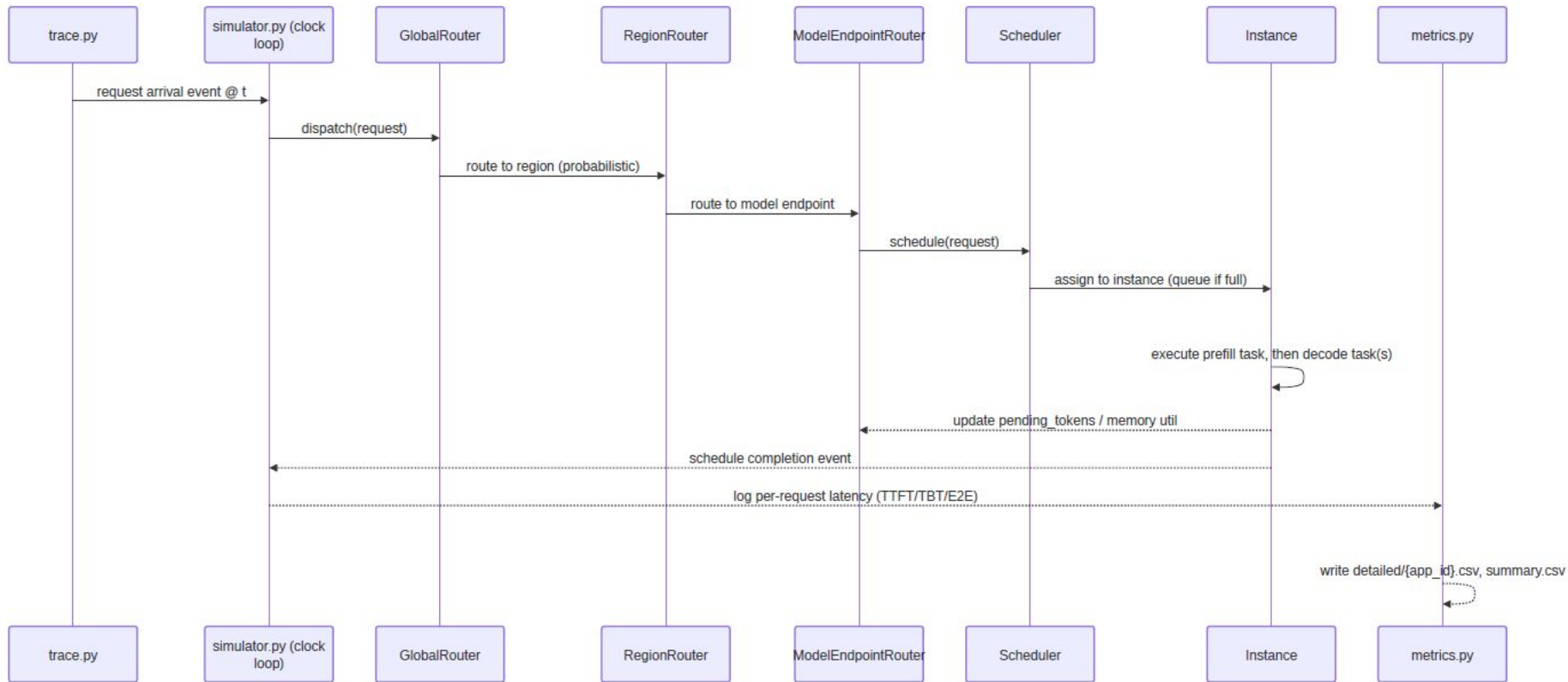


■ Decision logic (routing, scaling)
 ■ Simulated system
 ■ Foundations

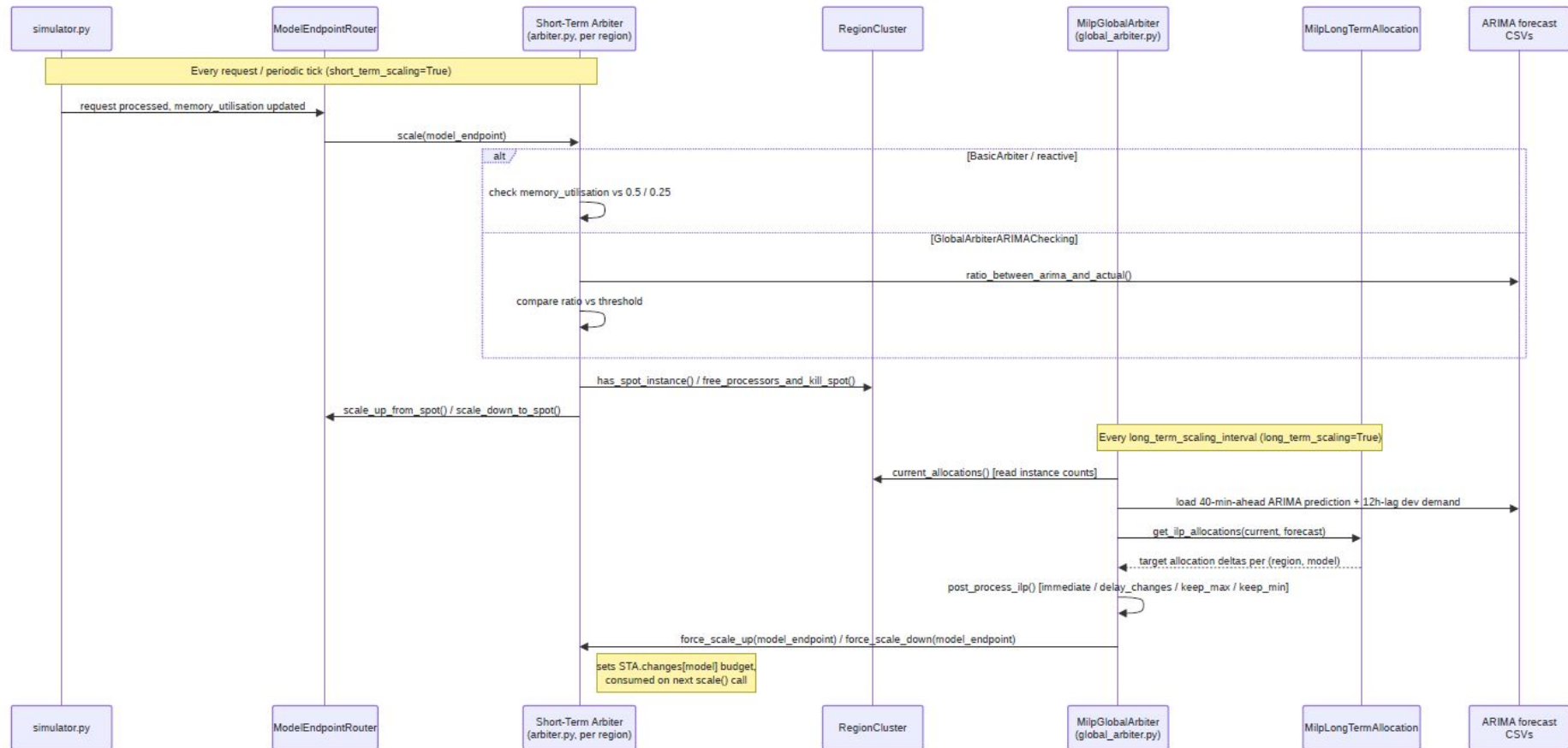
Down arrow: uses the level below. Side arrow: main link inside a level.

Group	Mathematical formulation
Resource Management	$\text{Workload Demand}(t) \rightarrow \text{Required Instances}(t) \rightarrow \text{Allocated Instances}(t)$
Scaling	$\text{Current Load}(t), \text{Forecast Load}(t) \rightarrow \text{Target Instance Count}(t)$
Allocation	$\text{Target Instance Count} - \text{Current Instance Count} \rightarrow \text{Instances to Start/Stop}$
Request Routing	$\text{Request} \rightarrow \text{Selected Region} \rightarrow \text{Selected Endpoint} \rightarrow \text{Selected Application}$
Infrastructure	$\text{Region} = \{\text{RegionCluster}\}, \text{RegionCluster} = \{\text{Server}, \text{Link}\}$
Application Instances	$\text{Application} = \{\text{Instance}_1, \text{Instance}_2, \dots, \text{Instance}_n\}$

End-to-End Request Flow in SageServe



AutoScaling: Short-Term Reactive + Long-Term Forecast-Aware



Test Setup: A Realistic Simulator Built on SplitWise

Simulator

- Extends SplitWise: discrete-event simulation of prefill + decode phases.
- MAPE < 3% vs. real hardware runs ($R^2 = 0.99$ prefill, 0.83 decode).
- Adds: unified event queue, region routing, multi-instance iteration.
- 4 open-source models: Bloom-176B, Llama2-70B, Llama3.1-8B, Llama3.2-3B on 8× A100/H100.

Scale of Evaluation

- 10M production requests replayed
- 3 US regions (East/Central/West)
- 4 open-source LLMs
- 2 week-long traces (Nov'24, Jul'25)

Progress

Understand

- Read the complete **SageServe** paper and studied:
 - Architecture and request flow
 - Reactive vs. forecast-aware scaling
 - ARIMA forecasting
 - ILP-based GPU provisioning
 - DPA request scheduling
 - Evaluation methodology
- Studied the **SplitWise/SageServe simulator internals** and documented the module flow.

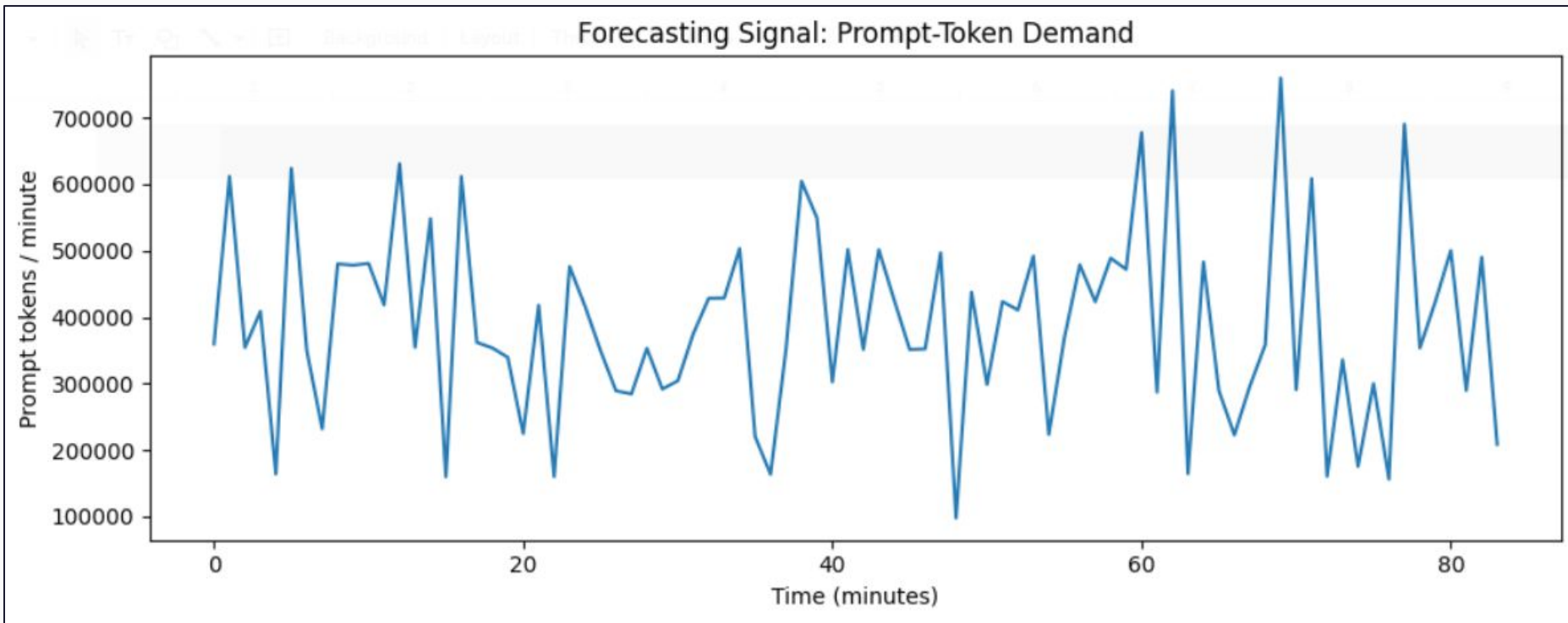
Reproduce the Environment

- Cloned the **original SageServe repository** into our project repository.
- Resolved the original repository's local execution issues:
 - Python 3.11 environment
 - Local launcher instead of the original Ray cluster setup
 - Configuration/constructor compatibility issues
 - Synthetic trace generation for local testing

Identified that SageServe is a **modified version of the SplitWise simulator**.

Random Trace Analysis: Prompt-Token Demand

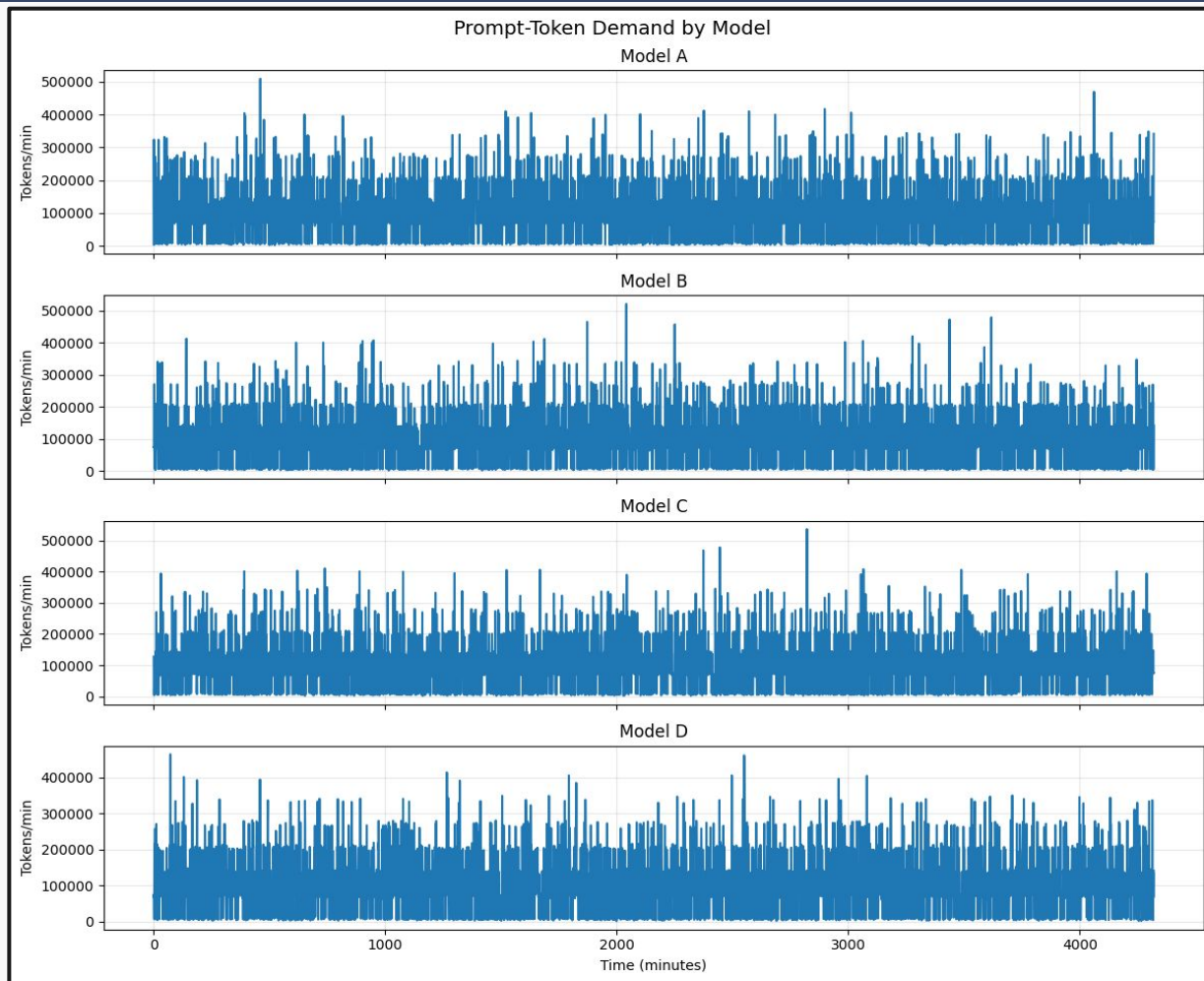
5000 requests , about 83 mins



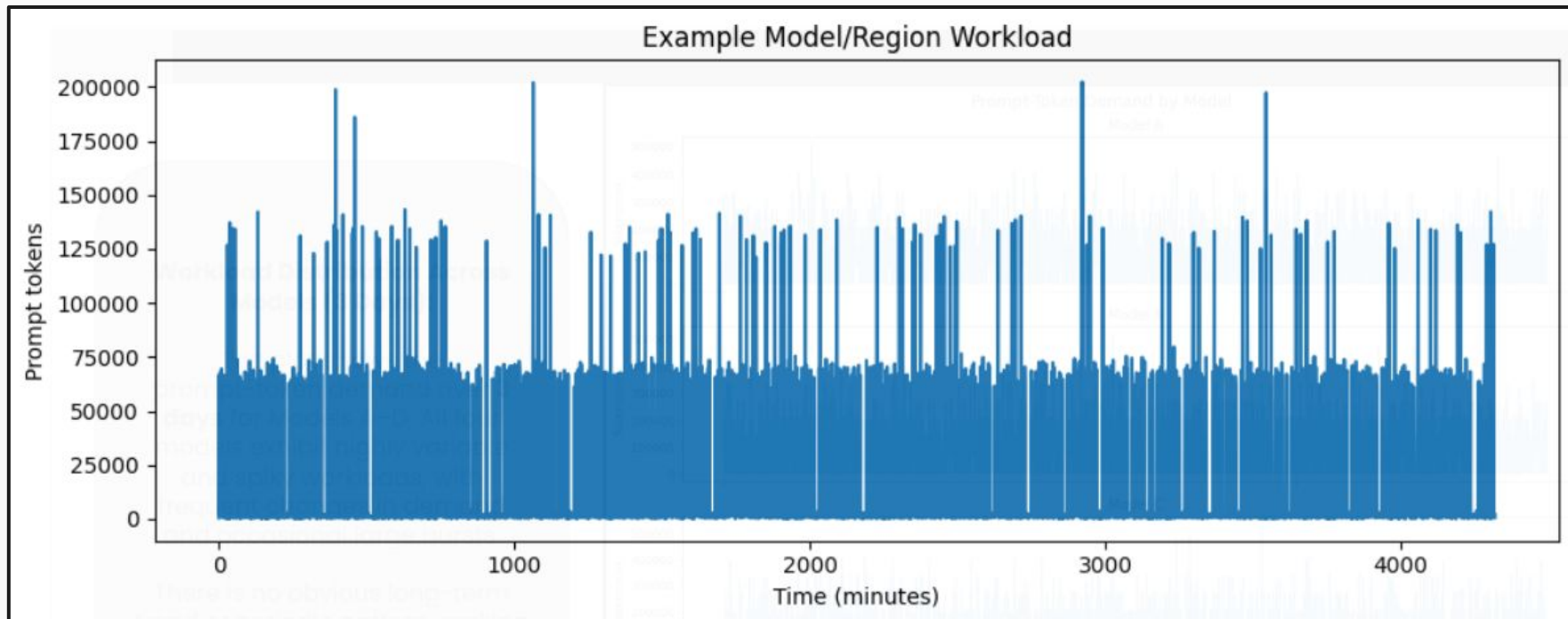
Workload Distribution Across Models (3 Days):

The plot shows the prompt-token demand over **3 days** for Models A–D. All four models exhibit highly variable and spiky workloads, with frequent changes in demand and occasional large bursts.

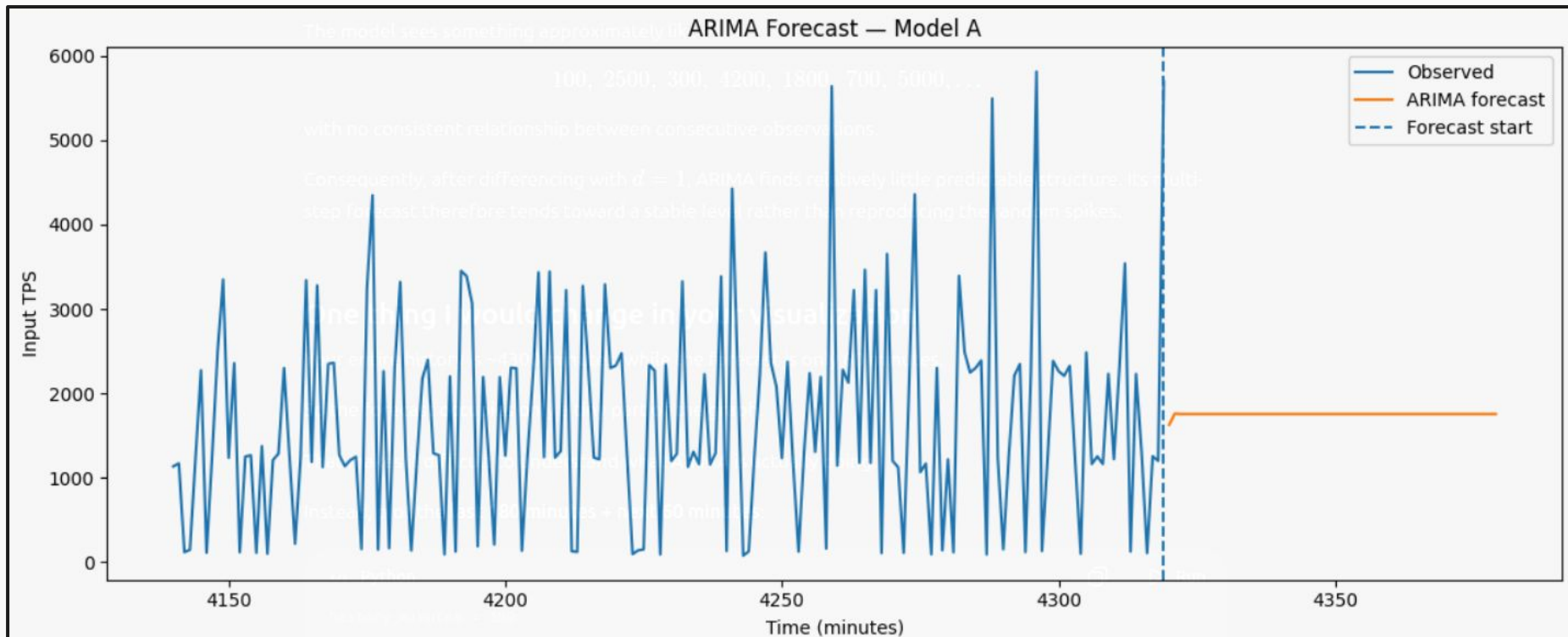
There is no obvious long-term trend or periodic pattern, making the workload difficult to predict directly and motivating the use of time-series forecasting such as ARIMA.



TPS for Model per Region



ARIMA Forecast for Model A



Progress and Next Steps

Run & Analyze

- Successfully got the **SageServe simulator running locally on Ubuntu.**
- Generated and executed **synthetic request traces.**
- Added notebooks/scripts to:
 - Inspect generated traces
 - Forecast preparation

Next Steps

- **Reproduction of paper results ?**
- **ARIMA forecasting**
- **Reactive vs. forecast-aware comparison Extension**
- **Cost analysis**

UNABLE TO REPRODUCE THE PAPER RESULTS AS THE PRODUCTIONS TRACES ARE NOT AVAILABLE